FAQ-17: How Do I Plan a Migration Cutover?

> **Series:** FAQ — Frequently Asked Questions | **Reference:** 17 — Planning a Migration Cutover | **Created:** July 2026 | **Last Updated:** 11/06/2026

Overview

Every migration in this corpus — Managed → SaaS, SaaS → SaaS, Splunk, New Relic, Sumo Logic, Classic Logs → OpenPipeline, management zones → policies — ends in the same shape. Different sources, different tooling, the same eight moving parts.

This entry names those parts and points at the series that documents each one best. **It restates none of them.** If you are running one of those migrations, your series already has the detail; use this to check you have not skipped an element. If you are writing a new migration runbook, use it as the outline so you do not re-derive the structure from scratch.

The eight invariants

#	Element	Skipping it means
1	**Go/No-Go readiness gate**	Cutover starts on a date rather than on evidence
2	**Validation ladder**	"It parses" gets mistaken for "it is correct"
3	**Parallel-run window with an end date**	The window never closes and both systems are maintained forever
4	**T-minus runbook**	Sequencing decisions get made on cutover day
5	**Rollback triggers, defined before the procedure**	The abort decision gets made under pressure by whoever is most tired
6	**Decommission sequence**	The old system lingers, still costing money and still authoritative to someone

#	Element	Skipping it means
7	**Stabilization window**	Cutover is declared done while the tuning backlog is still growing
8	**Lessons-learned capture**	The next migration in your estate starts from zero

Table of Contents

1. Where Each Element Is Documented
2. The Validation Ladder
3. The Parallel-Run Window
4. The T-Minus Skeleton
5. Rollback: Triggers Before Procedures
6. Entity-Level Parity Checks
7. Decommission and the Stabilization Window
8. If You Are Writing a New Migration Series
9. Summary and Next Steps

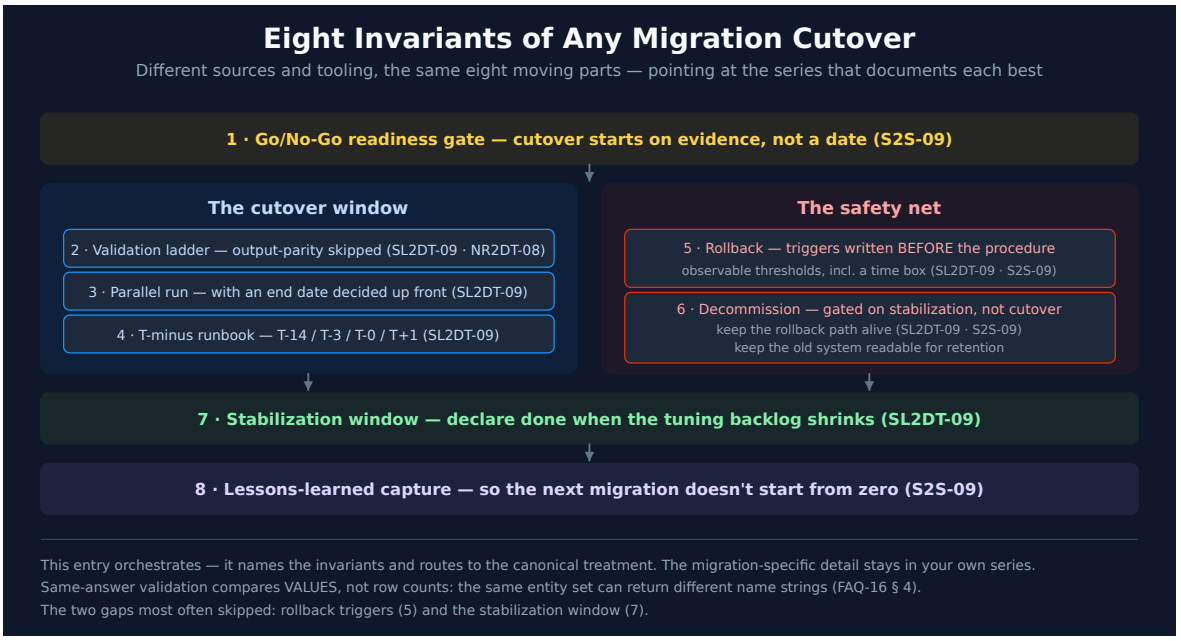
Prerequisites

Requirement	Details
Applies to	Any migration that ends in a cutover — tool replacement, deployment change, or in-place surface migration
Prior reading	The procedural series for your specific migration — this entry assumes it, it does not replace it
Permissions	Read on the tables you intend to validate; `storage:smartscape:read` for the entity-parity queries in section 6

> **Validation status.** The entity-parity queries in section 6 were executed against a live Dynatrace tenant on 07/23/2026, and re-executed on 09/28/2026 and 10/02/2026

> after the staleness checks gained an explicit `from:`. The rest of this entry is structural guidance and cross-references rather than DQL.

1. Where Each Element Is Documented



Two notebooks carry most of the weight. **SL2DT-09** is the most complete cutover runbook in the corpus; **S2S-09** has the strongest readiness gate and lessons-learned framework. Neither is Sumo- or tenant-specific in its structure, so both are worth reading whichever migration you are running.

Element	Read this first	Also see
Go/No-Go readiness gate	S2S-09 — Infrastructure / Configuration / Security / Data readiness	MZ2POL-06 (per-phase checklists)
Validation ladder	SL2DT-09 (three tiers); NR2DT-08 (syntax → tenant → output parity)	OPMIG-09 (per-concern validation checklist)
Parallel-run window	SL2DT-09 — setup, dual-ingest patterns, discipline rules, daily checklist	S2D and NRLC for tool-specific dual-run notes
T-minus runbook	SL2DT-09 — T-14 / T-3 / T-0 / T+1 with escalation criteria	S2S-09 (cutover timing recommendations)

Element	Read this first	Also see
Rollback	SL2DT-09 (triggers, runbook, post-rollback, anti-patterns)	S2S-09 (rollback by component)
Decommission	SL2DT-09 (pre-checks, sequence, data archival)	S2S-09 (decommission timeline)
Stabilization window	SL2DT-09 — first 30 days, week by week	ADOPT-05 (optimization roadmap)
Lessons learned	S2S-09 — capture template and common lessons	M2S-99 (framework synthesis)

For the migration-specific detail — what to inventory, what translates, what does not — go to your own series: M2S, S2S, NR2DT/NRLC, S2D, SL2DT, OPMIG, or MZ2POL.

2. The Validation Ladder

Two series document a three-tier validation model, but they cut the tiers at different places. The underlying ladder has four questions; each series places them in its own tiers:

Rung	The question it answers	NR2DT-08	SL2DT-09
Runs	Does it parse and run?	Tier 1 — Syntax	Implicit in Tier 1
Runs *here*	Does it run against real data in the target tenant, without erroring?	Tier 2 — Tenant	Implicit in Tier 1
Same answer	Does it produce the **same answer** as the thing it replaces?	Tier 3 — Output parity (behavioral)	Tier 1 — Technical parity (sampled query results, ingest volume, monitor firing, alert routing)
Accepted	Do the people who use it accept it?	—	Tier 2 — Functional equivalence; Tier 3 — Business signoff

Read the tier numbers inside each series; compare rungs across them. **The same-answer rung is the one that gets skipped**, because the rungs before it are automatable

and it usually is not. It is also the only rung that catches the failure mode that matters: a query that runs perfectly and returns different numbers.

A worked example of exactly that lives in FAQ-16 § 4 — a classic and a Smartscape query return **the same four entities** with **different `name` strings**, because classic host names carry a `[host-group] -` prefix that Smartscape names do not. Row counts match; every downstream string match breaks. The "runs" and "runs here" rungs both pass.

Compare values, not row counts. A count-only parity check is a "runs" check wearing a "same answer" costume.

> **Sources:** SL2DT-09 (three-tier validation model — its Tier 1 includes sampled query-result parity), NR2DT-08 (three-tier validation pass), FAQ-16 § 4 (the name-prefix divergence, verified against a live tenant 07/23/2026).

3. The Parallel-Run Window

Running old and new side by side is the validation mechanism, not a hedge. It is also the single most common place a migration stalls permanently.

Set the end date when you open the window, not when you feel ready to close it. A parallel run with no committed end date reliably becomes the steady state: both systems maintained, both partially trusted, every incident starting with an argument about which one is right. That costs more than either end state.

Three rules that travel across every migration type:

Rule	Why
One system is authoritative at a time, and everyone knows which	Two authorities means no authority during an incident
The window closes on evidence, not on elapsed time	"Four weeks" is a budget, not a criterion
Divergences are logged and triaged, not explained away	An unexplained divergence is an unvalidated migration

SL2DT-09 has the operational detail — dual-ingest patterns, discipline rules, and a daily validation checklist for the window. The same doctrine appears from the other direction in

the -START-HERE- playbook's Doorway 4, where a surface migration with no cutover date is the defining risk.

4. The T-Minus Skeleton

The generic shape, drawn from SL2DT-09. Adjust the intervals; keep the ordering.

Phase	What happens	What must be true to proceed
T-14	Pre-cutover prep; freeze scope; confirm rollback path is tested, not just written	Rollback has been *exercised* at least once
T-3	Final validation pass; Go/No-Go gate; stakeholder confirmation	All same-answer (output-parity) checks pass or are explicitly waived, in writing
T-0	Cutover; alerting silence window; escalation criteria live	Named owner present with authority to call rollback
T+1	Post-cutover validation; first-day parity checks	Same checks as T-3, re-run against the new authority
T+30	Stabilization complete; decommission proceeds	Tuning backlog is shrinking, not growing

The alerting silence window is worth naming explicitly. Cutover generates exactly the signal pattern that alerting exists to catch — entities disappearing, ingest volumes shifting, error rates moving. Silencing without a scheduled un-silence is how a migration ends with alerting quietly off in production.

5. Rollback: Triggers Before Procedures

Most rollback plans document the *procedure* and leave the *decision* implicit. That inverts the difficulty. The procedure is mechanical and can be written calmly in advance; the decision is a judgement call made at 02:00 by whoever is most invested in not rolling back.

Write the triggers first, and make them observable. A trigger is not "if things look bad." It is a threshold someone can check without debate:

Trigger shape	Example
Data loss exceeding a stated bound	Ingest volume on a migrated source below X% of baseline for Y minutes
Coverage regression	Entity count for a type below the pre-cutover figure by more than N
Detection regression	A previously-firing detection silent through a window where it should have fired
Time-box breach	Cutover not complete by T+H — roll back regardless of apparent progress

The last one is the one people leave out and need most. Without a time box, a partially-complete cutover extends indefinitely because each next step always looks like the shorter path.

SL2DT-09 documents triggers, runbook, post-rollback steps, and anti-patterns; S2S-09 covers rollback broken out by component.

6. Entity-Level Parity Checks

Data-level parity is migration-specific — your series has the queries. Entity-level parity is not: any migration that moves or re-instruments monitored things can be checked the same way — with one caveat about where the "before" run happens.

- **SaaS → SaaS.** Run the same query on both tenants with the same `from:`.
- **Managed → SaaS.** Before the first "before" run, check the source cluster with the DTM-99 health-check checklist, so the baseline is taken from a healthy cluster. `smartscapeNodes` is a Smartscape-on-Grail command, and in community practice the Managed source offers no equivalent DQL surface. Take the "before" inventory from the Managed environment's Monitored entities API v2 (`GET /api/v2/entities` with `entitySelector=type("HOST")`, and so on per type) and compare it per type with the target's `smartscapeNodes`. Expect differences between the classic and Smartscape counts that are not regressions: on the validation tenant (10/02/2026), the classic host list over a 2-hour window returned 15 hosts against 7 Smartscape `HOST` nodes on the same tenant. Treat a type-level gap as a prompt to reconcile entity by entity, not as a verdict.

Take this inventory before cutover and again after. A type whose count drops is a coverage regression, whatever the migration was:

```
// Entity inventory by type — run before cutover, save it, run again after.
// A type that drops is a coverage regression regardless of migration type.
// Uses the default timeframe (nodes seen recently) — keep the timeframe
// identical between the before and after runs.
smartscapeNodes "*"
| summarize entity_count = count(), by:{type}
| sort entity_count desc
```

Then check for entities that have stopped reporting. `lifetime[end]` is the last time Smartscape saw the node, so anything stale relative to now is a candidate for something that did not survive the move:

```
// Hosts that stopped reporting — the post-cutover check that catches
// agents which were moved but never came back. The from: is required:
// a node not seen inside the query timeframe is not returned at all.
smartscapeNodes "HOST", from:-7d
| fieldsAdd last_seen = lifetime[end]
| fieldsAdd stale = last_seen &lt; now() - 1h
| summarize hosts = count(), by:{stale}
| sort stale asc
```

Widen it to name the specific stragglers once the count tells you there are some:

```
// Name the stragglers so they can be chased individually.
smartscapeNodes "HOST", from:-7d
| fieldsAdd last_seen = lifetime[end]
| filter last_seen &lt; now() - 1h
| fields id, name, last_seen, dt.host_group.id
| sort last_seen asc
```

> **On the timeframe.** `smartscapeNodes` returns only nodes seen inside the query timeframe, and the default is short. Without `from:`, a host that went silent before the window started is **not returned at all**, so the check reports "none stale" exactly when hosts have been lost. Set `from:` to reach back before the cutover began — on the validation tenant (09/28/2026) the default window returned 7 hosts, none stale; `from:-7d` returned the same 7 plus **4 stale hosts** last seen three days earlier.
>
> **On thresholds.** `now() - 1h` is a starting point, not a recommendation. Set it from your own reporting interval and the length of your cutover window — too tight and

> normal restarts look like losses, too loose and you finish cutover before the check can tell you anything.
>
> **Sources:** all three queries executed against a Dynatrace tenant, 07/23/2026 — the inventory returned 15+ node types led by BROWSER_MONITOR_STEP (1,342), CONTAINER (437), PROCESS (353); the staleness check returned 11 hosts, none stale. Re-executed 09/28/2026: without `from:`, 7 hosts none stale; with `from:-7d`, 7 current + 4 stale (last seen 09/25/2026). Re-executed 10/02/2026: with `from:-7d`, 7 current + 6 stale; `fetch dt.entity.host, from:-2h` returned 15 hosts against 7 from `smartscapeNodes "HOST"`, no notifications.
>
> **Sources:** Monitored entities API - GET entities list (DT Managed docs) — the Managed endpoint `https://{your-domain}/e/{your-environment-id}/api/v2/entities`, which *"Lists entities observed within the specified timeframe along with their properties"*, Smartscape on Grail (DT docs).

7. Decommission and the Stabilization Window

Decommission is gated on the stabilization window, not on cutover. The old system stays available — not authoritative, but available — until the new one has been steady for a defined period. Retiring it at T-0 removes the rollback path exactly when it is most likely to be needed.

Three things to settle before the old system goes away:

Question	Why it bites later
What historical data must be retained, and where?	Retention obligations rarely migrate with the tooling; SL2DT-09 covers data archival
Who else consumes the old system's output?	Reports, exports, and integrations outlive the team that built them
What is still licensed?	Cost savings are the business case, and they do not land until decommission does

Retention is met by keeping the old system readable, or by an archive outside Dynatrace — not by re-ingesting history. Dynatrace log ingest drops records older than its log age limit, which is measured in hours: 72 by default for environments created after SaaS 1.348, and 24 for older environments unless Dynatrace Support raises it to 72 on

request. Either way it is far short of a multi-year obligation. Replaying an archive through a Collector does not get round it, because replay keeps each record's original timestamp.

SL2DT-09 breaks the stabilization window into weeks — stabilize, tune, hand off, retro — with metrics to track through it. That structure generalizes; the metrics will not.

> **Sources:**
>
> - Log Management and Analytics default limits (DT docs) — *"The event is dropped if it contains a timestamp older than the configured log age limit." "For environments created after Dynatrace version 1.348 was released, the default log age limit is 72 hours." "For environments created before Dynatrace version 1.348 was released, the default log age limit is 24 hours. The limit can be increased to 72 hours on request."*
> - What's new in Dynatrace SaaS 1.348 (DT docs) — *"The log ingestion pipeline now accepts log records with timestamps up to 72 hours in the past,"* and *"Rollout start on Oct 12, 2026"*
> - **Code read (10/29/2026):** `internal/blobconsume/consumer.go` at tag `v1.2.0`, used by the S3, Azure Blob and Google Cloud Storage rehydration receivers, unmarshals each stored OTLP payload and passes it on unchanged (lines 105–114), so replayed records keep their original timestamps

8. If You Are Writing a New Migration Series

Use the eight invariants as the outline for the final step, and cite this entry plus the canonical treatments rather than restating them. Three things are worth getting right:

1. **Name the same-answer check explicitly.** Every series documents the mechanical rungs well. Say what output parity means for *your* migration, and what evidence closes it.
2. **State the parallel-run end condition, not just its length.** "Two weeks" is a budget. "Zero unexplained divergences across three consecutive daily checks" is a criterion.
3. **Put rollback triggers before the rollback runbook,** in that order on the page. The ordering signals which one gets read under pressure.

What belongs in your series rather than here: the inventory, what translates cleanly, what needs redesign, the tool-specific validation queries, and the source-system decommission mechanics.

9. Summary and Next Steps

Four things to carry away:

1. **Eight invariants, every migration.** Check yours against the list; the gaps are usually 5 (rollback triggers) and 7 (stabilization).
2. **The same-answer rung is the one that matters** — and comparing row counts is not it.
3. **The parallel-run window needs an end date decided at the start.** Otherwise it becomes the architecture.
4. **Decommission gates on stabilization, not cutover.** Keep the rollback path alive through the window.

If you need…	Read
The most complete cutover runbook in the corpus	SL2DT-09 (cutover, validation, decommission)
The strongest readiness gate and lessons-learned template	S2S-09 (optimize)
Validation tiers for a translation-heavy migration	NR2DT-08 (validate)
Per-concern validation for a pipeline migration	OPMIG-09 (troubleshooting and validation)
Phase-gated execution for an access-model migration	MZ2POL-06 (migration execution)
Why output parity beats row-count parity	FAQ-16 § 4
Which doorway your migration sits in	The -START-HERE- playbook

---

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.